

Rethinking Compact (<1M) Vision Models: Balancing Accuracy and Speed through Multi-Path Atrous Convolutions

Christos Kyrkou

KIOS Research and Innovation Center of Excellence

University of Cyprus

1 Panepistimiou Avenue, Nicosia, Cyprus

kyrkou.christos@ucy.ac.cy

Abstract

Deep learning has transformed computer vision, yet state-of-the-art models increasingly rely on large-scale architectures and datasets, reinforcing a “scale is all you need” paradigm. This trend creates challenges for real-time, resource-constrained systems, where even lightweight models often contain millions of parameters. This work, systematically investigates the landscape of deep learning models in the sub-1M parameter regime, for small- to medium- sized datasets representative of vision tasks (classification and detection), and provides empirical comparison of representative architectures, highlighting cross-dataset generalization under strict parameter budgets. Notably, it is observed that naive downscaling of existing architectures often leads to inconsistent performance, and that Vision Transformers underperform relative to CNNs in data-constrained, low-parameter settings. To address these gaps, this work proposes a novel architecture specifically tailored for the sub-1M regime that utilizes multi-path atrous convolutions to capture contextual features efficiently. The proposed model outperforms existing compact architectures even at 100K parameter regime, while delivering on average $3.3\times$ faster inference on real devices. These results demonstrate that small-scale models, when architecturally rethought rather than merely reduced, can achieve both efficiency and competitive performance in constrained environments.

1. Introduction

Deep Neural Networks (DNNs) have been at the forefront of continual and significant advances on problems such as image classification [11, 38] and object detection [6, 19, 49]. More recently with the use of Transformer networks [43] and Self-Supervised Learning [1] more focus has been given towards operating on large compute clusters, and models with billions of parameters and proprietary large-scale datasets.

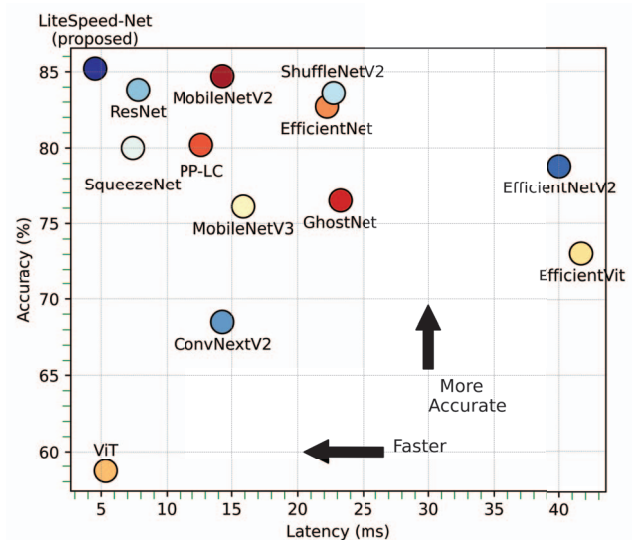


Figure 1. Comparing the accuracy-speed of different sub-1M parameter vision models. Latency tested on Jetson Orin with batch size of 1 and accuracy on ImageNet-16 dataset.

As a result, one of the key emerging challenges has become how to optimize AI models for devices with limited resources, such as IoT, smartphones, and autonomous drones [41],[27]. Known as edge AI, the goal of this area is to permit such devices to perform vision tasks autonomously without relying on cloud computing. However, implementing deep learning models on these edge systems requires addressing constraints like computation time, energy consumption, memory footprint, model size, and even carbon emissions [42][33][26]. Hence, developing smaller and efficient models tailored to small- and medium-sized datasets is essential to enable AI capabilities in these resource-constrained environments [4, 25].

Smaller models with fewer parameters and efficient operations are more amicable to running on devices with limited

resources [10, 45]. This allows their deployment in environments including edge computing and decentralized systems [34]. Lightweight models allow for easier scaling and running many AI tasks in parallel. More importantly, smaller models can also lead to quicker inference times, making them ideal for real-time applications. These advantages make it worthwhile to explore the architectural landscape of small efficient models and how they balance accuracy and speed and which is more suitable for edge AI computing platforms such as embedded GPU accelerated platforms [12],[51]. While prior research has compared large-scale pretrained backbones [11], small neural networks suitable for edge deployment (e.g., IoT, robotics, wearables) with fewer than 1 million parameters remain underexplored on small-to-medium datasets [3]; compared to mobile-scale models 2-20 million parameter models.

This work is motivated by the absence of systematic evaluation and design principles for effective small-scale vision models and how they perform in terms of accuracy and inference time within current architectural paradigms, motivated by the practical constraints of limited compute and modest data availability. It is argued that architectural design, i.e., how parameters are structured and utilized, matters as much as their quantity, necessitating a systematic evaluation of current architectures [11, 13, 32, 37, 40] in the low-parameter regime. Building on these insights, this work introduces LiteSpeed-Net (LS-Net)¹, a novel lightweight architecture that leverages multi-branch structures with atrous depthwise convolutions to efficiently capture contextual features. While this work builds on multi-scale and depthwise principles it is the first to systematically analyze their performance in the sub-1M-parameter regime. Empirical evaluations on both standard, and real-world benchmarks (in supplementary material (SM)) show that LS-Net achieves a superior accuracy-latency trade-off. It delivers up to a +2% accuracy gain with a 3.3× speedup in image classification, and on average runs 1.5× faster in detection tasks while maintaining competitive robustness and generalization across datasets and device classes. Notably, these advantages persist even when the model is scaled down to as few as 100K parameters. These results position LS-Net as a strong candidate for edge AI applications requiring compact, high-speed, and accurate models.

This study reveals several key insights: ① The performance of existing models in terms of accuracy varies unpredictably across datasets, indicating that scaling alone does not generalize well in the low-parameter regime. Rather, the low-parameter regime requires architectural rethinking. ② While vision transformers (ViTs) are increasingly popular they still lag behind convolutional neural networks (CNNs) in the small-to-medium parameter/data regimes, which provides opportunities for further research toward

this area. ③ Architectures optimized for low Multiply-Accumulate Operations (MACs) may underperform in terms of real latency. ④ Incorporating multi-scale feature fusion with atrous depthwise convolutions enables consistently improved performance and faster computation.

2. Architectural Trends

Increasing Depth with Residuals: As CNNs grew deeper, they encountered issues with vanishing and exploding gradients. He et al. [14] addressed this with ResNet, which introduced residual connections to alleviate these problems, enabling much deeper networks. The core ResNet architecture uses identity shortcut connections across convolutional or attention layers to bypass degradation in performance as depth increases since they enable models to retain useful information[31],[28].

The Depthwise Separable Convolution Paradigm: In 2017, MobileNet [15] was explicitly proposed to provide lightweight and efficient neural network models that can operate in real-time on devices with limited memory, and constrained compute. The model utilized depthwise separable convolution, which involves the decomposition of a traditional convolution into two distinct operations: depthwise and pointwise convolution using a 1×1 kernel. In conventional convolution, kernels are collectively applied to all the input channels, resulting in a single combined output. Furthermore, depthwise separable convolution independent channel-wise convolution is applied before mixing this with a point-wise convolution. The authors also suggested the integration of two hyperparameters, precisely the width and resolution multipliers, to optimize the computational efficiency and reduce the model’s dimensions. Sandler et al.[37] introduced an improved version MobileNetV2 using the main concepts of MobilenetV1 such as depth-wise separable convolutions and model parameter reduction. The MobileNetV2 architecture introduced a novel module called the inverted residual with a linear bottleneck (IRB), which aimed to decrease the computational requirements and enhance accuracy. It first expands the feature space using a pointwise convolution and then does spatial feature extraction with a depthwise separable convolution before it is projected back to a lower dimensional space. The MobileNetV3 model [20] builds on top of the depth-wise separable paradigm and inverted residual bottleneck through a combination of hardware-aware network architecture search techniques and novel architecture designs. Specifically, the architectural parameters are obtained by neural architecture search (NAS) [50]. It also uses the H-swish non-linearity which has been proved experimentally to improve accuracy. In addition, it adds a squeeze and excitation module to help in adjusting the per channel weights of different feature maps. The work in [8] pushes the limits of depthwise convolutions further and proposes optimizations for designing lightweight

¹Code Repository: <https://github.com/ckyrkou/videl>

CNNs, by improving the activation function, selective use of squeeze-and-excitation blocks [16], and larger convolutional filters.

Compound Scaling: The EfficientNet [40] architecture was designed to balance high efficiency and accuracy. It investigated the effect of scaling of depth, width, and resolution, and proposed the use of a combined scaling factor to adjust the three dimension together. This helps to control the target computational complexity (i.e., MACs) of a given DNN model. The B0 version (baseline) was introduced as the smallest and lightest version among other versions of EfficientNet, suitable for devices with limited resources. However, as demonstrated in this work the number of parameters alone and the MACs/FLOPs metrics might not directly translate to efficient operation on device. To address these limitations, EfficientNetV2 [39] adopts a progressive learning strategy that increases image resolution and data augmentation complexity over time. Second, it improves network design by incorporating two blocks, MBConv (MobileNet block) and Fused-MBConv which uses a regular 3×3 convolution and leverages neural architecture search to automatically search for the best combination.

Shuffling: ShuffleNetV2 [32] has been developed for speed in mind rather than FLOPs like EfficientNet. It improves the architecture of ShuffleNetV1, by ensuring that i) the input and output channels have equal widths, thus minimizing the cost associated with memory access, ii) makes targeted use of group convolution to reduce computational demands, iii) improved bottleneck by removing element-wise operations such as addition. Based on these it proposes a novel building block where the input is divided into two segments using a channel split layer and the channel shuffle operation is moved further down the block.

Squeezing: The SqueezeNet [17] model was developed to enable efficient neural network inference on devices with limited resources, such as mobile devices. The architecture includes a combination of 1×1 and 3×3 which incur the least amount of computation compared to those with larger receptive field. The primary use of 1×1 convolutions is to increase the number of channels, enhancing the potential to capture intricate patterns. The use of 3×3 convolutions facilitates the acquisition of spatial information and contributes to the acquisition of hierarchical features. Drawing inspiration from ResNet more recent versions incorporate residual connections. This adaptation resulted in improved accuracy compared with the original model.

Cheap Operations: GhostNet [13] proposes a ghost module that first uses standard convolutional operation to obtain a few intrinsic features and then generates "ghost features" through cheap linear operations (e.g., depthwise convolutions). This leads to rich features with minimal computational cost, resulting in maintaining accuracy with fewer parameters and computations.

Attention and Patches: The latest architectural involves using image patches as tokens, which are processed through the self-attention mechanism in transformer networks [9]. This Vision Transformer (ViT) architecture also includes straightforward feed-forward and layer normalization layers. While the architecture of the vanilla ViT seems simple and potentially efficient, the removal of inductive biases makes it challenging to train them from scratch for smaller datasets and retain competitive performance [24]. Typically ViT models rely on large parameter counts and extensive pretraining on large-scale datasets which makes their adoption in edge AI systems challenging [26]. Another challenge is with ViT's global attention design is its quadratic complexity relative to the input size which has been the subject of recent studies that proposed alternatives such as cascaded group attention [29]. The recent advances of Vision Transformers has motivated researchers to explore similar concepts for ConvNets. The authors in [30] propose ConvNeXt by modernizing a ResNet towards a hierarchical Transformer-like structure by adding a patchify layer as well as incorporating larger kernels, and layernorm normalization. This resulted in better scalability and accuracy, without sacrificing the efficiency and simplicity of ConvNets. More recently, ConvNeXtV2 [46] introduced several key improvements to allow ConvNets to better work more effectively in a self-supervised setting.

Main Limitations: Existing paradigms remain constrained by (1) limited sampling diversity due to repeated local receptive fields, (2) quadratic compute growth when expanding receptive fields via larger kernels, and (3) weak correlation between FLOPs and actual device latency. Furthermore, prior work does not quantify the trade-off between distinct spatial samples and computational cost.

3. LiteSpeed Deep Network Architecture

Motivated by the aforementioned observations, this work presents a compact model design that reallocates parameters to enhance spatial diversity through parallel execution paths yielding more favorable efficiency–accuracy trade-offs. The hypothesis is that as parameter budgets shrink, the ability to learn diverse spatial representations becomes increasingly critical for discrimination, requiring effective contextualization of local features. Guided by this principle, the LiteSpeed architecture is introduced.

3.1. Multi-Path Atrous Convolutions

At the core of the LiteSpeed Network is the **Multi-Path Atrous Convolutions (MPAC)** block. It is a generic block that can be configured to include different operations and aims to: i) Extract multi-scale features to capture both local details and more global context simultaneously. ii) Aggregate and increase feature dimensionality through concatenation which does not incur additional computational

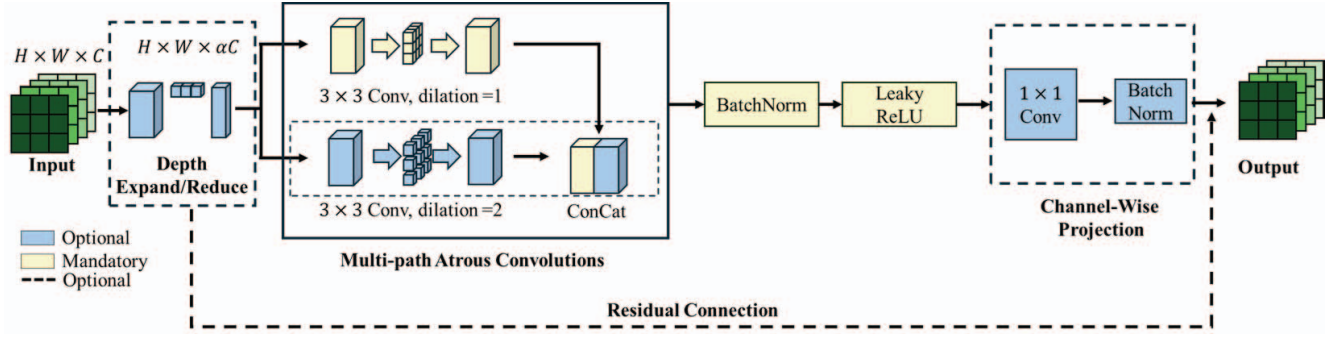


Figure 2. Multi-Path Atrous Convolution (MPAC) Block. This block is repeated within the general architecture and instantiated with different configurations and convolutional channel dimensions. In this work this block is instantiated without the depth expander/reducer.

cost. iii) Exploit atrous convolutions that skip input values leading to larger receptive field with fewer operations to cover a larger area, providing a richer representation. iv) Utilize depth-wise convolutions for all the atrous convolutions to learn spatial features in a computationally-efficient way. Unlike similar blocks found in ASPP[5]/Inception[47], which target large models or use full convolutions or use dilated convolutions after a backbone, MPAC is explicitly constrained to minimal branching, depthwise atrous operations, and concatenation-driven channel expansion across all layers. The main MPAC operation can be summarized as follows.

Let $X \in \mathbb{R}^{H \times W \times C}$ denote the input tensor. The MPAC block aims to maximize receptive field coverage and add more context. To achieve this, the block applies M parallel depth-wise atrous convolutions with distinct dilation factors d_i . For the i -th path (with simplified notation):

$$Y^{(i)}(l, m) = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W_{u,v}^{(i)} * X_{l+d_i u, m+d_i v}, i = 1, \dots, M \quad (1)$$

where $Y^{(i)} \in \mathbb{R}^{H \times W \times C}$ is the output feature map of path i , $W^{(i)} \in \mathbb{R}^{k \times k}$ are depthwise kernels of path i , d_i denotes the convolution dilation rate. The multi-scale responses are then concatenated along the channel axis:

$$Z = \text{Concat}(Y^{(1)}, Y^{(2)}, \dots, Y^{(M)}) \in \mathbb{R}^{H \times W \times (MC)} \quad (2)$$

A block with 2 atrous convolutions and concatenation is implemented as follows:

$$\begin{aligned} Y^{(1)} &= \text{AtrousDepthConv}_{d_1}(X) \\ Y^{(2)} &= \text{AtrousDepthConv}_{d_2}(X) \\ Z &= \text{concat}([Y^{(1)}, Y^{(2)}]) \end{aligned}$$

The $\text{AtrousDepthConv}(\cdot)_{d_i}$ block refers to the i th block of atrous convolution operations with dilation rate of $d_i = i$ (e.g., d_1 is a canonical 3×3 kernel and $d_2 = 2$ (i.e., effective

5×5 receptive field with 9 parameters). $\text{concat}[\cdot, \cdot]$ refers to the channel-wise feature aggregation. In this case concatenating features from different atrous convolutions allows diverse representation of the input that can improve discrimination ability. Note that the block can also be instantiated with only the $d = 1$ atrous convolution effectively becoming a VGG-style block. This operation is followed by a normalization operation and activation which in this case are batch normalization and LeakyRelu respectively, the latter selected for its higher efficiency, improved gradient propagation, and support in many inference deployment platforms [29].

There are optional operations that can be instantiated within the block. Prior to the main MPAC operation a channel expansion/reduction module, controlled by the non-negative parameter α , can be instantiated to transform the input tensor such that $\mathbb{R}^{H \times W \times C} \rightarrow \mathbb{R}^{H \times W \times (\alpha \cdot C)}$, which is akin to an inverted residual block as in [37] when $\alpha > 1$ or reduction block as in [17] when $\alpha < 1$. Finally, at the end a projection component is used to mix the feature maps using 1×1 convolution. The input can optionally be added to the output tensor to form a residual block to aid in gradient propagation. Overall, the MPAC block encapsulates various architectural choices resulting in a highly adaptable, efficient, and powerful building block that can be used to generate efficient small deep learning models.

3.2. Network Architecture

Generic Architecture: The architecture utilizes a stem block, followed by a series of stacked MPAC blocks utilizing strided convolutions for downsampling instantiating a ~ 1 million parameter model described herein. Subsequent smaller model sizes can be derived by applying a scaling factor on the number of filters (shown in Section 4.4).

1M Model Instantiation: The stem layer is a standard convolutional layer which outputs 24 filters with batchnorm normalization and also reduces the spatial resolution by 2. Then a series of MPAC blocks are followed by gradually

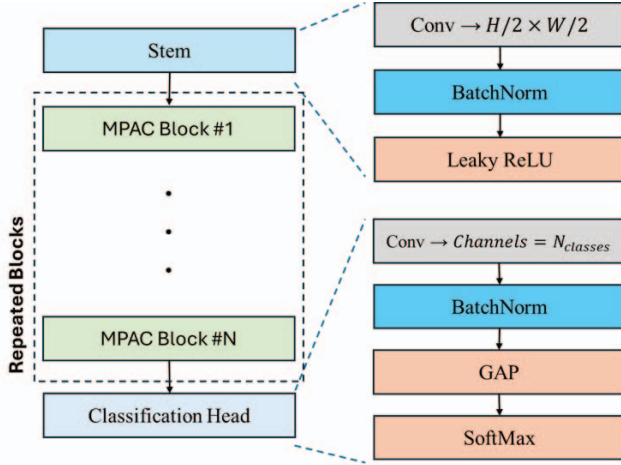


Figure 3. Generic Architecture of LiteSpeed Network.

increased channels by adding powers of 2, grouped into stages according to the size of the feature maps that they progress. Downsampling is applied at the beginning of each stage, allowing subsequent blocks within that stage to process features with the same or an increased number of filters. The number of channels and number of blocks were chosen through empirical design exploration and then subsequently used for all datasets and experiments. The last convolutional block followed by a normalization layer, projects the output feature map from the preceding stage to the number of classes. Then a global average pooling is utilized to get the final representation vector for classification. Overall, batch normalization is used instead of layer normalization, as seen in recent Transformer architectures, to better leverage larger batch sizes made feasible by the smaller model sizes. More details on the architecture and theoretical insights are provided in the SM (S1 and S2).

4. Experiments and Results

To evaluate performance, the proposed model is compared with state-of-the-art models used for image classification as outlined in Section 2. These include MobileNets [20, 37], ShuffleNetV2 [32], SqueezeNet² [17], EfficientNet [40], EfficientNetV2 [39], ConvNextV2 [46], GhostNet [13], PP-LC [8], ResNet³ [14], original Vision Transformers⁴ [9] and efficient variants [29]. All models are scaled down so that their parameters are less than 1 million as shown in Table 2 by adjusting a channel width multiplier factor. More results on additional datasets, robustness, ablations, and training settings are provided in the SM (S5,S6,S7,S8,S9).

² Added Batchnorm to base model

³ ResNet18 variant, 4 layers with blocks per layer set as 4,3,2 and 1.

⁴ Depth of 4 and number of heads is 3. Patch size is 10 for IIC, 4 for C10, and 16 for ImgN16

<1M Param Models	Acc-ImgN16 (%) ↑	Acc-C10 (%) ↑	Acc-IIC (%) ↑
<i>MobileNetV2</i> [37]	84.7	83.5	91.1
<i>GhostNet</i> [13]	76.5	75.6 [†]	89.0 [†]
<i>PP-LC</i> [8]	80.2	75.6 [†]	90.4
<i>EfficientNet</i> [40]	82.7	84.4	89.0 [†]
<i>EfficientNetV2</i> [39]	78.8	84.6	89.6 [†]
<i>ViT</i> [9]	58.75 [†]	74.4 [†]	84.6 [†]
<i>EfficientViT</i> [29]	73.0	84.1	88.9
<i>ConvNextV2</i> [46]	68.5 [†]	78.7 [†]	88.3 [†]
<i>MobileNetV3</i> [20]	76.1 [†]	72.2 [†]	88.8 [†]
<i>SqueezeNet</i> [17]	80.0 [†]	68.2 [†]	90.1 [†]
<i>ShuffleNetV2</i> [32]	83.6	79.2	89.7
<i>ResNet</i> [14]	83.8	91.3	91.3 [†]
<i>LiteSpeed-Net</i> (proposed)	86.5	91.9	91.6

Table 1. Comparison of sub-1M parameter Vision Networks on different datasets

[†] Best results with initial learning rate of $1e-3$

4.1. Benchmark Suite

To perform an investigation of the architectural landscape and assess the effectiveness of the proposed model, experiments are conducted using a benchmark suite of three small-to-medium classification datasets. These include the CIFAR-10 [21] dataset (referred to as C10), Intel Image Classification (referred to as IIC) dataset⁵, and the ImageNet-16 [22] dataset (referred to as ImgN16), a subset of the ImageNet dataset. Collectively, these datasets provide a comprehensive evaluation set, encompassing a wide range of class counts and image resolutions, thereby enabling a thorough assessment of the proposed approach. Further details on each dataset are available in the SM-S4.

4.2. Accuracy Results

Accuracy results of the proposed LS-Net model are presented in Table 1 and compared with scaled-down versions of established architectures all trained from scratch. Additionally, the proposed network is compared to other works in the literature, that do not specifically focus on smaller models and also evaluated on larger datasets.

Firstly, for the ImageNet-16 dataset LS-Net achieves the highest accuracy at 86.5%, demonstrating a strong performance in this parameter budget. Specifically, it surpasses the MobileNetV2 model by 1.8% which is the second best and has a large margin on other models such as ResNet and ShuffleNetV2. More specialized architecture such as PP-LC and GhostNet fall further behind. Notably, the ViT architecture provides the lowest accuracy, while the ConvNext model also performs less effectively.

⁵ available on Kaggle <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>

Comparisons on the CIFAR-10 dataset, which has the most number of samples from the benchmark suite, again demonstrate that the proposed LS-Net provides the higher accuracy, followed by the ResNet model and then EfficientNetV2. Note that for this lower resolution dataset the previously second model MobileNetV2 drops to fourth. This demonstrates that the proposed architecture is even competitive at the lower resolution regime where features become more difficult to detect. Notably, only LS-Net and ResNet reach over 90% accuracy on the CIFAR-10 dataset. As a reference, conventional multi-million parameter models that do not make use of advanced methods like learnable augmentations, ensembling, self-supervision, or large-scale pretraining generally achieve accuracies in the 90 – 95% range [22]. The proposed LS-Net performs competitively, at a significantly smaller size. More details on varying settings and results for this dataset are presented in SM-S9.

Finally, when comparing the different models on the Intel Image Classification challenge dataset, where it is necessary to understand the whole scene and not focus on specific objects within the image, the proposed model again manages to provide the highest accuracy. It is followed by ResNet and MobileNetV2. When comparing with some existing models from the literature the proposed LS-Net compares favorably. Notably it is also able to provide higher accuracy than some established multi-million parameter models that also utilize transfer learning [36],[47] which exhibit accuracies ranging from 87%-94%. It also compares favorably to other networks (2-3 million parameters) reported in the literature [44], but with a fraction of the parameters while being trained from scratch.

4.3. Inference Speed

While the accuracy results may suggest that the different models are closely matched, it is essential to consider the inference speed as well, especially for real-time edge AI systems. Therefore, accuracy should be evaluated in conjunction with inference speed to gain a comprehensive understanding of model performance. The average frames-per-second (FPS) for each model is reported after 10,000 iterations of profiling. Table 2 presents the performance comparison of the small vision models considering the two different computing platforms common for EdgeAI evaluation [2], NVIDIA Jetson AGX Orin and NVIDIA Jetson Nano (more details in SM-S3). Number of model parameters, MACs, and FPS performance on both devices are also provided, allowing for a clear evaluation of the speed advantages of the proposed LS-Net. Additional results are shown in SM-S10.

LS-Net demonstrates a clear speed advantage on the NVIDIA Jetson Orin, achieving over 200 FPS, which is the highest among all the models. ResNet achieves 128 FPS, which is 41% slower than LS-Net. ViT reaches 187

<1M Param Models	Parameters ($\times 10^3$)	MACs** ($\times 10^6$)	FPS [†] (1/s) ↑	FPS [‡] (1/s) ↑
MobileNetV2 [37]	~ 956	164	70	25
GhostNet [13]	~ 962	45	43	12
PP-LC [8]	~ 906	97	79	22
EfficientNet [40]	~ 987	115	45	15
EfficientNetV2 [39]	~ 900	220	25	8
ViT [9]	~ 980	186	187	57
EfficientViT [29]	~ 932	34	24	8
ConvNextV2 [46]	~ 967	324	70	23
MobileNetV3 [20]	~ 995	40.9	63	18
SqueezeNet [17]	~ 967	947	135	30
ShuffleNetV2 [32]	~ 944	97	44	14
ResNet [14]	~ 973	3190	128	12
LiteSpeed-Net (proposed)	~ 919	295	220	64

Table 2. Comparison in terms of FPS of the small models

[†] Measured for a resolution of 224×224 on NVIDIA Jetson Orin

[‡] Measured for a resolution of 224×224 on NVIDIA Jetson Nano

** MACs estimated using: <https://github.com/sovrasov/flops-counter.pytorch/tree/master/ptflops>

FPS, slightly trailing LS-Net by approximately 18% but with much lower accuracies. SqueezeNet achieves 135 FPS, which is slower by ~ 38%. Other models like MobileNetV2 (70 FPS), MobileNetV3 (63 FPS), and EfficientNet (45 FPS) show significantly lower performance, with LS-Net being up to 3–4 times faster. On the Jetson Nano, LS-Net continues to outperform the competing models, achieving 64 FPS. Again, this is the highest across the models. ViT comes closest with 57 FPS, but LS-Net still maintains a 12% higher FPS achieving much higher accuracies. Other models like ResNet (12 FPS), SqueezeNet (30 FPS), MobileNetV2 (25 FPS), and MobileNetV3 (18 FPS) perform significantly worse, with LS-Net being 2-5 times faster.

LS-Net has 295 million MACs, placing it towards the lower end of the spectrum when compared to more computationally expensive models like ResNet (3190 million MACs) and SqueezeNet (947 million MACs). While some models like GhostNet (45 MACs) and MobileNetV3 (40.9 MACs) have fewer MACs than LS-Net, their FPS performance is substantially lower (e.g., GhostNet at 43 FPS on Jetson Orin and 12 FPS on Jetson Nano). Despite higher MACs, LS-Net outperforms these models in terms of FPS, demonstrating its ability to achieve better utilization. Specifically, while it shows increase in MACs, these are devoted to 1×1 convolutional operations which are highly parallelized on modern EdgeAI devices. This consistent performance advantage across both computing platforms suggests that the proposed architectural considerations translate into faster real-time processing. LS-Net not only excels in scenarios requiring real-time processing but is also able to consistently outperform other small models with strong accuracy performance across datasets.

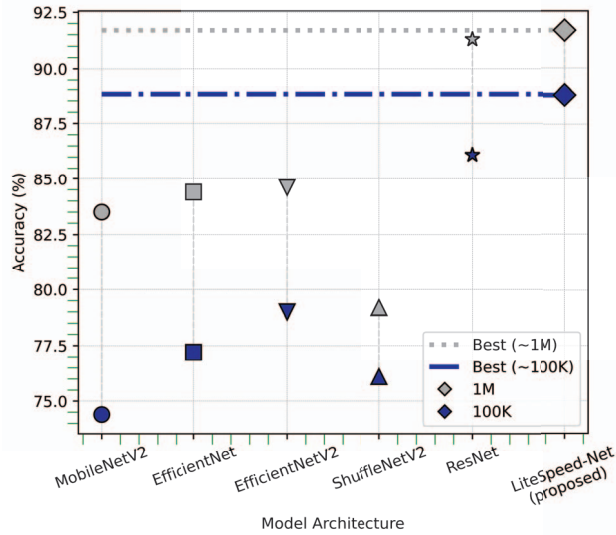


Figure 4. Performance of models on CIFAR-10 when scaling down to $\sim 100K$ parameters and comparison with $\sim 1M$ versions. (marker shapes represent model, color represents size, Best viewed in color)

4.4. Scaling Down to Tiny Models

This experiment, investigates the scaling trends when reducing parameters to the order of $\sim 100K$ for tiny models. These models are better suited for on-device training and data privacy, particularly on lower-end AI and IoT devices and have seen increasing adoption in real-world applications [23]. However, deploying established pretrained models in these environments presents challenges due to limited memory and computational resources. By analyzing the behavior of various architectures under parameter constraints, it is possible to investigate which structures are more resilient to scaling down.

First, the top-performing models from the previous experiments are selected, specifically those that ranked among the top four across the datasets, namely: *LS-Net*, *ResNet*, *MobileNetV2*, *EfficientNetV2*, *EfficientNet*, and *ShuffleNetV2*. Then they are scaled down until they reached just over 100K parameters without any other modification, and compare them on the CIFAR-10 and ImageNet-16 datasets, with results presented in Table 3.

Initially it is observed that the proposed LS-Net achieved the higher accuracy in the tiny model regime, maintaining a significant advantage over the rest. Also, it is observed that the ranking of the different models is not consistent across datasets. For instance, ResNet has the second highest accuracy for CIFAR-10 but is ranked fifth on ImageNet-16. This demonstrates another strength of LS-Net in that it is capable of maintaining competitive performance across datasets for different scales. Additionally, the performance of the tiny

$\sim 100K$ Param Models	Params ($\times 10^3$)	Acc-ImgN16 (%) $\uparrow$	Acc-C10 (%) $\uparrow$
<i>MobileNetV2</i> [37]	~ 154	80.1	74.4
<i>EfficientNet</i> [40]	~ 147	80.6	77.2
<i>EfficientNetV2</i> [39]	~ 140	78.8	84.6
<i>ShuffleNetV2</i> [32]	~ 151	81.0	76.1
<i>ResNet</i> [14]	~ 114	79.1	86.1
<i>LiteSpeed-Net</i> (proposed)	~ 145	81.4	88.8

Table 3. Performance of scaled down versions of best models to tiny sizes of $\sim 100K$ parameters.

models is compared with their 1M-parameter counterparts. As illustrated in Fig. 4, the tiny models generally show a decline in performance, which is expected, with most dropping below 80% accuracy for CIFAR-10, making them less competitive and practical. Interestingly, the proposed 100K version of the LS-Net model outperformed almost all the other 1M parameters models for CIFAR-10, also demonstrating the least drop from the 1M counterpart.

4.5. Additional Comparisons and Insights

Trends on Larger-Scale Datasets: Beyond the comparative study, LS-Net is positioned in the context of larger-scale datasets, which introduce greater complexity due to the increased number of classes/instances, as well as comparing with additional models. To evaluate its scalability, the same sub-1M LS-Net model is trained on TinyImageNet and a subset of ImageNet with 100 classes⁶. These datasets provide a natural progression in complexity while maintaining a manageable scale for low-parameter models. The 1M parameter LS-Net achieves 46% top-1 accuracy on TinyImageNet, which is on par with 2M parameter WaveMix models and comparable to ResNet models [18] in the literature. Additionally, our 100K model outperforms all reported sub-1M WaveMix models on CIFAR-10 [18]. When tested on the Orin, a 1M parameter WaveMix model achieves 40 FPS, whereas LS-Net reaches 200 FPS, highlighting its ability to deliver both accuracy and speed in low-parameter settings. On the ImageNet subset, the proposed model achieves 70% top-1 accuracy, positioning it competitively against significantly larger models [48].

Why LS-Net works: It is observed that overall LS-Net demonstrates significant speedup over other models, but the specific advantage varies depending on the model and device. Indicating that the hardware architecture influences the model performance. Speedups range from 1.1 $\times$ for the Vision Transformer to 5.3 $\times$ for ResNet and GhostNet. However, the ViT model trails significantly in terms of accuracy. LS-Net shows notable speedup over models considered more

⁶<https://www.kaggle.com/datasets/ambityga/imagenet100>

mobile-friendly like MobileNetV2 (2.5× on Orin, 3.1× on Nano) and MobileNetV3 (3.5× on Orin, 3.4× on Nano). The consistent speedup of the proposed model suggests that its specialized architecture can adequately accommodate different hardware as its parallel receptive fields provide more feature reuse and higher FPS. Specifically, LS-Net incorporates design principles of early downsampling, fewer filters in initial layers, simple operations (depthwise convolutions without the expansion as in MobileNets), no complex operations (e.g., squeeze-excite as in EfficientNet), smaller dilated kernels (compared to full 7×7 used in ConvNextV2), and Leaky ReLU activation (compared to exponential functions). The LS-Net model maintains consistently high accuracies in the low-parameter regime compared to other models that are scaled down from larger architectures, as it is built bottom-up with efficiency in mind with the mixing of multi-scale features. This ensures better use of learning capacity as parameters are distributed more effectively across layers.

4.6. Object Detection

Expanding to tasks such as object detection would necessitate investigating aspects such as neck and head architectures and additional loss functions. For simplicity typical structure for head and neck are set, and parameters, and use IoU-based loss functions for box regression. All models were trained from scratch. The utilized datasets come from the Roboflow 100⁷ benchmark suite [7] for animals, vehicles, and construction safety, the HomeObjects-3K⁸ dataset for applications in indoor scene understanding and smart home, and for fine-grained detection the person/person-like VOC [35] subset on Kaggle⁹.

Results are provided in Table 4 for object detection comparing sub-1M model backbones, with added detection neck and heads. Results demonstrate that the proposed LS-Net achieves the highest or equal mAP@0.5 across all categories, outperforming existing models in Vehicles (38.5%), Animals (60.5%), Construction (81.5%), Person/Person-like (75.4%) and Home Objects (54.4%). While some models, like EfficientNet and ShuffleNetV2, perform well in specific classes, their results are less consistent. ResNet shows strong detection for Vehicles but under-performs significantly for Animals, highlighting its limitations. Overall, LS-Net offers the most balanced and accurate performance across diverse object types. More comparisons and results are reported in SM (S11, and S12).

The FPS performance of the best performing models for object detection were evaluated using the Jetson Orin platform. The LS-Net model demonstrated the highest throughput at 50 FPS, due to its optimized architecture. The ResNet

<1M Param Backbones	Vehicles	Animals	Construction	Person	Home Objects
	mAP@0.5 (%) ↑				
<i>MobileNetV2</i> [37]	37.4	42.1	75.1	70.5	44.6
<i>EfficientNet</i> [40]	38.5	50.9	76.8	60.7	39.7
<i>EfficientNetV2</i> [39]	36.3	39.3	73.5	63.9	51.8
<i>ShuffleNetV2</i> [32]	37.6	46.4	70.2	69.7	50.0
<i>ResNet</i> [14]	38.1	40.8	73.1	65.2	49.1
<i>LiteSpeed-Net (proposed)</i>	38.5	60.5	81.5	75.4	54.4

Table 4. Object Detection Results

variant followed with 42 FPS, outperforming models such as MobileNetV2 which achieved 34 FPS. EfficientNet and ShuffleNetV2 showed moderate inference speeds of 26 and 24 FPS, respectively, EfficientNetV2 exhibited the lowest performance at 16 FPS. These results highlight the capability of the proposed architecture to provide speedups in the range of 1.3 – 3.8 coupled with high accuracy.

5. Concluding Remarks

This work investigated the landscape of sub-1M parameter neural network models in vision tasks. It further developed a block and overall architecture referred to as *LiteSpeed Network*, and experiments validated it achieving both high accuracy and high inference speed across small-to-medium datasets of different resolutions and number of classes, common characteristics of many real-world applications, making it well-suited for resource-constrained edge AI systems. This work is among the first to show systematic demonstrations that multi-path atrous convolutions can provide meaningful and consistent latency/accuracy trade-off in the small model regime. Furthermore, this work is orthogonal to quantization and pruning methods, and has set a competitive baseline for small vision models, especially when training from scratch, motivating more research in this direction. The findings also demonstrate that the recent trend of using Vision Transformers might not be suitable for low-parameter and small-to-medium data regimes, especially when training small models from scratch. Future research will integrate the MPAC block with automated model search methods, like NAS, to develop more efficient architectures at smaller scales.

Limitations: The insights are shaped by the specific datasets, architectures, and configurations used in this study. Nonetheless, a principled methodology was followed that provides valuable guidance for designing efficient neural networks while results under varying settings are presented in SM (S7,S8,S9). While it is possible that alternative architectures may perform competitively, the fact that the proposed architecture performs well without having to modify it for different datasets suggests, that it offers a strong, "off-the-shelf" baseline for research in fast and accurate small vision models.

⁷<https://universe.roboflow.com/roboflow-100>

⁸<https://docs.ultralytics.com/datasets/detect/homeobjects-3k/>

⁹<https://www.kaggle.com/datasets/karthika95/pedestrian-detection>

Acknowledgements

This work was supported by the European Union's Horizon Europe research and innovation programme under grant agreement No. 101168067 (GuardAI). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

The author would like to thank the reviewers for their valuable feedback, which strengthened the manuscript through additional experiments and comparisons.

References

- [1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023. 1
- [2] Stephan Patrick Baller, Anshul Jindal, Mohak Chadha, and Michael Gerndt. Deepedgebench: Benchmarking deep neural networks on edge devices. In *2021 IEEE international conference on cloud engineering (IC2E)*, pages 20–30. IEEE, 2021. 6
- [3] Bjorn Barz and Joachim Denzler. Deep learning on small datasets without pre-training using cosine loss. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 1371–1380, 2020. 2
- [4] Lihu Chen and Gaël Varoquaux. What is the role of small models in the llm era: A survey. *arXiv preprint arXiv:2409.06857*, 2024. 1
- [5] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proceedings of the European conference on computer vision (ECCV)*, pages 801–818, 2018. 4
- [6] Gong Cheng, Xiang Yuan, Xiwen Yao, Kebin Yan, Qinghua Zeng, Xingxing Xie, and Junwei Han. Towards large-scale small object detection: Survey and benchmarks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11): 13467–13488, 2023. 1
- [7] Floriana Ciaglia, Francesco Saverio Zuppichini, Paul Guerrie, Mark McQuade, and Jacob Solawetz. Roboflow 100: A rich, multi-domain object detection benchmark, 2022. 8
- [8] Cheng Cui, Tingquan Gao, Shengyu Wei, Yuning Du, Ruoyu Guo, Shuilong Dong, Bin Lu, Ying Zhou, Xueying Lv, Qiwen Liu, et al. Pp-lcnet: A lightweight cpu convolutional neural network. *arXiv preprint arXiv:2109.15099*, 2021. 2, 5, 6
- [9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. 3, 5, 6
- [10] Muhammad Fahim, S. M. Ahsan Kazmi, Vishal Sharma, Hyundong Shin, and Trung Q. Duong. Edge intelligence: A deep distilled model for wearables to enable proactive eldercare. *IEEE Transactions on Artificial Intelligence*, 6(7): 1736–1745, 2025. 2
- [11] Micah Goldblum, Hossein Souri, Renkun Ni, Manli Shu, Viraj Prabhu, Gowthami Somepalli, Prithvijit Chattopadhyay, Mark Ibrahim, Adrien Bardes, Judy Hoffman, et al. Battle of the backbones: A large-scale comparison of pretrained models across computer vision tasks. *Advances in Neural Information Processing Systems*, 36, 2024. 1, 2
- [12] Hui Han and Julien Siebert. Tinyml: A systematic review and synthesis of existing research. In *2022 International Conference on Artificial Intelligence in Information and Communication (ICAIIIC)*, pages 269–274. IEEE, 2022. 2
- [13] Kai Han, Yunhe Wang, Qi Tian, Jianyuan Guo, Chunjing Xu, and Chang Xu. Ghostnet: More features from cheap operations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 2, 3, 5, 6
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. 2, 5, 6, 7, 8
- [15] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017. 2
- [16] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7132–7141, 2018. 3
- [17] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and < 0.5 mb model size. *arXiv preprint arXiv:1602.07360*, 2016. 3, 4, 5, 6
- [18] Pranav Jeevan and Amit Sethi. Wavemix: resource-efficient token mixing for images. *arXiv preprint arXiv:2203.03689*, 2022. 7
- [19] Licheng Jiao, Ruohan Zhang, Fang Liu, Shuyuan Yang, Biao Hou, Lingling Li, and Xu Tang. New generation deep learning for video object detection: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 33(8):3195–3215, 2022. 1
- [20] Brett Koonce and Brett Koonce. Mobilenetv3. *Convolutional Neural Networks with Swift for Tensorflow: Image Recognition and Dataset Categorization*, pages 125–144, 2021. 2, 5, 6
- [21] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical Report 0, University of Toronto, Toronto, Ontario, 2009. 5
- [22] Christos Kyrkou. Toward efficient convolutional neural networks with structured ternary patterns. *IEEE Transactions on Neural Networks and Learning Systems*, pages 1–8, 2024. 5, 6
- [23] Christos Kyrkou and Theodoris Theodoridis. Emergencynet: Efficient aerial image classification for drone-based

- emergency monitoring using atrous convolutional feature fusion. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 13:1687–1699, 2020. 7
- [24] Seunghoon Lee, Seunghyun Lee, and Byung Cheol Song. Improving vision transformers to learn small-size dataset from scratch. *IEEE Access*, 10:123212–123224, 2022. 3
- [25] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, and Song Han. Tiny machine learning: progress and futures. *IEEE Circuits and Systems Magazine*, 23(3):8–34, 2023. 1
- [26] Di Liu, Hao Kong, Xiangzhong Luo, Weichen Liu, and Ravi Subramaniam. Bringing ai to edge: From deep learning’s perspective. *Neurocomputing*, 485:297–320, 2022. 1, 3
- [27] Li Liu, Matti Pietikainen, Jie Qin, Wanli Ouyang, and Luc Van Gool. Efficient visual recognition. *International Journal of Computer Vision*, 128:1997–2001, 2020. 1
- [28] Tianyi Liu, Minshuo Chen, Mo Zhou, Simon S Du, Enlu Zhou, and Tuo Zhao. Towards understanding the importance of shortcut connections in residual networks. *Advances in neural information processing systems*, 32, 2019. 2
- [29] Xinyu Liu, Houwen Peng, Ningxin Zheng, Yuqing Yang, Han Hu, and Yixuan Yuan. Efficientvit: Memory efficient vision transformer with cascaded group attention. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14420–14430, 2023. 3, 4, 5, 6
- [30] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022. 3
- [31] Jian-Hao Luo and Jianxin Wu. Neural network pruning with residual-connections and limited-data. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1458–1467, 2020. 2
- [32] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In *Proceedings of the European conference on computer vision (ECCV)*, pages 116–131, 2018. 2, 3, 5, 6, 7, 8
- [33] Sparsh Mittal, Poonam Rajput, and Sreenivas Subramoney. A survey of deep learning on cpus: Opportunities and co-optimizations. *IEEE Transactions on Neural Networks and Learning Systems*, 33(10):5095–5115, 2022. 1
- [34] Tamador Mohaidat and Kasem Khalil. A survey on neural network hardware accelerators. *IEEE Transactions on Artificial Intelligence*, 5(8):3801–3822, 2024. 2
- [35] Chandran Saravanan N. J. Karthika. Addressing false positives in pedestrian detection. In *International Conference on Electronic Systems and Intelligent Computing (ESIC 2020)*, 2020. 8
- [36] Mohammad Rahimzadeh, Soroush Parvin, Amirali Askari, Elnaz Safi, and Mohammad Reza Mohammadi. Wise-srnet: a novel architecture for enhancing image classification by learning spatial resolution of feature maps. *Pattern Analysis and Applications*, 27(2):30, 2024. 6
- [37] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 2, 4, 5, 6, 7, 8
- [38] Maneet Singh, Shruti Nagpal, Richa Singh, and Mayank Vatsa. Derivenet for (very) low resolution image classification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):6569–6577, 2022. 1
- [39] Mingxing Tan and Quoc Le. Efficientnetv2: Smaller models and faster training. In *International conference on machine learning*, pages 10096–10106. PMLR, 2021. 3, 5, 6, 7, 8
- [40] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *CoRR*, abs/1905.11946, 2019. 2, 3, 5, 6, 7, 8
- [41] Kristina Telegraph and Christos Kyrkou. Spatiotemporal object detection for improved aerial vehicle detection in traffic monitoring. *IEEE Transactions on Artificial Intelligence*, 5(12):6159–6171, 2024. 1
- [42] Gaël Varoquaux, Alexandra Sasha Luccioni, and Meredith Whittaker. Hype, sustainability, and the price of the bigger-is-better paradigm in ai. *arXiv preprint arXiv:2409.14160*, 2024. 1
- [43] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017. 1
- [44] Satvik Vats, Jai Prakash Bhati, Abhinash Singla, Vinay Kukreja, and Rishabh Sharma. Advanced image classification on intel datasets using optimized efficientnet and mobilenetv2. In *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, pages 1–4, 2024. 6
- [45] Chuan Wang, Qiang Liu, Yusheng Li, and Mingwang Gao. Lightcspnet: A lightweight network for image classification and objection detection. *International Journal of Computational Intelligence Systems*, 16(1):46, 2023. 2
- [46] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon, and Saining Xie. Convnext v2: Co-designing and scaling convnets with masked autoencoders. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16133–16142, 2023. 3, 5, 6
- [47] Xizhi Wu, Rongzhe Liu, Hanqing Yang, and Zizhao Chen. An xception based convolutional neural network for scene image classification with transfer learning. In *2020 2nd International Conference on Information Technology and Computer Application (ITCA)*, pages 262–267, 2020. 4, 6
- [48] Zelin Zang, Siyuan Li, Di Wu, Ge Wang, Kai Wang, Lei Shang, Baigui Sun, Hao Li, and Stan Z Li. Dlme: Deep local-flatness manifold embedding. In *European Conference on Computer Vision*, pages 576–592. Springer, 2022. 7
- [49] Zhong-Qiu Zhao, Peng Zheng, Shou-Tao Xu, and Xindong Wu. Object detection with deep learning: A review. *IEEE Transactions on Neural Networks and Learning Systems*, 30(11):3212–3232, 2019. 1
- [50] Xiawu Zheng, Chenyi Yang, Shaokun Zhang, Yan Wang, Baochang Zhang, Yongjian Wu, Yunsheng Wu, Ling Shao, and Rongrong Ji. Ddpnas: Efficient neural architecture search via dynamic distribution pruning. *International Journal of Computer Vision*, 131(5):1234–1249, 2023. 2
- [51] Yan Zhou, Shaochang Chen, Yiming Wang, and Wenming Huan. Review of research on lightweight convolutional neural networks. In *2020 IEEE 5th Information Technology*

and Mechatronics Engineering Conference (ITOEC), pages
1713–1720, 2020. [2](#)